Lab2 实验报告

Phase 1

外层 sscanf 取若干字段，再调用 p1_compute，其内部依次调用 p1_step0、p1_step1、p1_step2。challenge 在求解函数中被拆成低 32 位、>>16 段、高 32 位三个片段。

p1_compute 依次调用 p1_step0、p1_step1、p1_step2，三个子函数结构同型，均把传入值乘 3（lea (%rdi,%rdi,2)）再与一个常量异或。追 p1_compute 入口到各 call 之间参数寄存器的赋值位置，可确定 p1_step2 接收 high 与 low 两个参数，最终由两个中间值再做一次乘 3 异或得到答案，即 s0 =((low+low*2+mid)^0x53fe)，其中 low=(u32)challenge、mid=(u32)(challenge>>16)；s2=((high +low*2)^0x767c)，其中 high=(u32)(challenge>>32)，答案=(s0+s0*2)^(s2+0xb3f7)。

Phase 2

要求 6 个整数；校验函数先用 sscanf 逐字段解析读入 6 个整数，a[0]单独校验，随后进入一个以 i 为下标的循环。a[0]由(challenge>>7)&0xf 加常量得到，随后 a[1]~a[5]用循环递推。循环体内每轮再从 challenge 取一个新的 4 位半字节参与运算，移位量从 0x1b 起步、每轮增加 5；同时异或的常量含循环下标偏移(i+0xe)，并把上一项乘 5（lea (%rax,%rax,4)）后累加，形成递推链：a[0]=((challenge>>7)&0xf)+0x11　对 i=1 至 5，shift 从 0x1b 每轮+5；a[i]=(((challenge>>shift)&0xf)^(i+0xe))+5*a[i-1]

Phase 3

challenge&7 作为选择子，跳到 8 个 case 之一；表 0x405820 的 8 个 8 字节条目读出，与 nm 中 p3_case_*的符号地址逐一比对，得到选择子到 case 体的乱序映射（序号与下标并非直线对应）。每个 case 体是对 x=(challenge>>3)^(challenge>>32)的一次小变换，形态为乘小常数再加/异或常量，可选右移，形式主要有：5x+C、4x^C、(x^C)+(x>>k)、3x+C 等。

Phase 4

函数接收四个参数，查找目标 target、区间端点 lo、hi，以及一个参与返回值混合的 key。区间初值为[0, 0x7f]，即只在 0~127 中查找。递归推进取中点 mid=(lo+hi)/2，出口为 lo>hi 时返回(mid+0xa6)^key；若 target 小于 mid 则递归左半区间，返回值再异或常量；否则递归右半区间并作另一常量异或。目标取 challenge 的低 7 位，key 取两个移位视野的异或。用 static 辅助函数实现，求解函数把最终返回值写入 values[]，并令 count≥1，调用参数为 target=challenge&0x7f、key=(challenge>>7)^(challenge>>32)。

Phase 5

静态节点表以{next, A, B}（4 字节字段）布局，另有一张头节点下标表；节点表按 4 字节宽度 dump 后与各字段一一对应。求解时先从 0x405b60 处的指针表由 challenge 选出头节点，累加器初值由 challenge 两个移位视野的异或、首节点的 B 与常量 0x462a 混合而成；行走步数沿 next 由 challenge 的相邻 3 位决定，即 steps=(challenge>>3)&3。每走一步执行指针前进-累加器异或该节点 B-累加器加上该节点 A，并记录最后节点的 A。输出累加器终值与末节点 A。

Phase 6

challenge&7 和(challenge>>3)&7 各 3 位分别作为下标，从函数指针表 0x405e00 中取函数调

用，同一张表被调用两次，第一次的返回值用^0x8f94 串联，一次常量异或后作为第二次的输入，两次结果与 challenge 的混合值一起参与最终加法。主控把一个 64 位值传给 op 函数，op 内部以 mov%edi,%eax、xor%eax,%edi 等 32 位操作处理，参数实际按 32 位参与全部运算。函数指针表在 0x405e00，用 x/8gx 读出全部条目，主控以 challenge&7 和 (challenge>>3)&7 两次查表。六个 op 函数逐一反汇编并串联两次调用，得到 op_0：(2x+0x3fa8)^5x；op_1：((x>>5)^x)+0xe2f9；op_2：(x+0xdb2a)^(x>>3)；op_3：(x^0x63a0)+4x；op_4：((x<<2)|(x>>30))^0xbf2a（rol $2 后异或）；op_5：(x>>2)|(x<<30)（ror$2，无异或）。设 x=(challenge>>6)^(challenge>>32)；则有 r1=p6_op(challenge&7, x)；r2=p6_op((challenge>>3)&7, r1^0x8f94)；答案=r2+0x7074。表中下标 1 与下标 4 指向同一函数，故实现时这两个分支共用同一个表达式。中间值 r1/r2 可用 GDB 在两次调用返回后读 $eax 逐一核对。

Honor 关

求解接口中 context 由前六行口令经 transcript_context 得出。先依次调用六个求解函数、用它们的输出算出 context 并自动传入求解函数。nm 找到四个相关函数，secret_phase(0x402650)、secret_gate(0x402730)、secret_mix(0x402630)、secret_copy(0x402710)，以及 honor_started 标志与 secret_agent_decoy/secret_agent_urgent 字符串。数据流方向从调用链上确定：secret_gate 解析并分发，secret_phase 校验，secret_mix 运算，secret_copy 复制文本。还原 secret_copy 函数体，逐字节复制循环说明第七行是自由格式文本，会被原样拷入校验缓冲区。还原 secret_mix，反汇编得到运算 mix(x)=((rol32(x,3))+0xb1b8)^0xcd5f。还原 secret_gate 的格式约束，拷贝后用一条 4 字节比较校验偏移 0x20 处等于 0x4e45504f，小端字节序还原出 OPEN；通过后以行+36 为首地址调用 secret_phase，并把两个 64 位值分别经 rsi、rdx 传入，由此确定第七行=32 字符前缀+OPEN+三个无符号整数 A、B、C。还原 secret_phase 校验，先 valid_u32_fields(rol,3)确认是三个合法无符号十进制段，再 sscanf 读入 A（偏移-0x2c）、B（-0x28）、C（-0x24）。两个 64 位输入经 main 调用链确认为 rsi=challenge、rdx=context，校验逻辑用 32 位折叠值表达，设 e1=(u32)((challenge>>32)^challenge)；e2=(u32)((context>>32)^context)；则三个条件为 A=(u32)context&3；B=mix(e1^e2)^((0xb1b8*A)^(u32)(context>>16))；C=B^((e1>>3)+e2+A+0xcd5f)。A 只由 context 的低 2 位决定，第二、三条逐步叠加 mix 与常量。实现时由 A 推 B、由 B 推 C 正向构造。

kyt813@kk: ~/ics-2026-bombl

BOMBLAB 2026　　这里还没有明天



[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

一个普通的晚上，你和林夏在实验室接上了一台旧设备，
却误入了一座陌生的数字城市。住在这里的，是 AI。
街边的维修铺还开着，老板正低头摆弄一盏灯。
再往前走，维护站的门半开着，里面只有一台旧终端。
屏幕上留着一条消息：

  没想到这里还会有人来。
  祝你们好运。
  —— sky

窗口角落留着‘返回实验室’的按钮。
林夏把外套搭在椅背上：‘先看看？’
维护面板上画着一枚炸弹，旁边亮着六项未完成的交接检查。
它不是街上的爆炸物，只是旧系统给这次维护选的图标。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

```
Phase 1 challenge: 0x543afafccc6521b8
Input (unsigned decimal integers):
Phase 1 defused.
```

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

```
[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x6029b85c0842f06f
Input (unsigned decimal integers):
Phase 2 defused.

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。
```

```
[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x4e2e0ab0c568b112
Input (unsigned decimal integers):
Phase 3 defused.

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。
```

```
[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x969582ddf95ce66b
Input (unsigned decimal integers):
Phase 4 defused.

最后一层索引返回了一份完整存档，还有那次迁移留下的记录。
新设备接上时，交接没有做完。城市只能反复恢复最后一份旧存档；
居民今天做过的事，到了夜里就都没了。
存档后面列着几处还没核对的设备。你认出了维修铺那盏灯。
```

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 当前] [6 待解除]

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x8cbcc7bdfc3a62e7
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。
```

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0xaaef50d6243fe594
Input (unsigned decimal integers):
Phase 6 defused.

终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
```

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

谢谢你远道而来，与我相遇。

晚安。

—— sky
